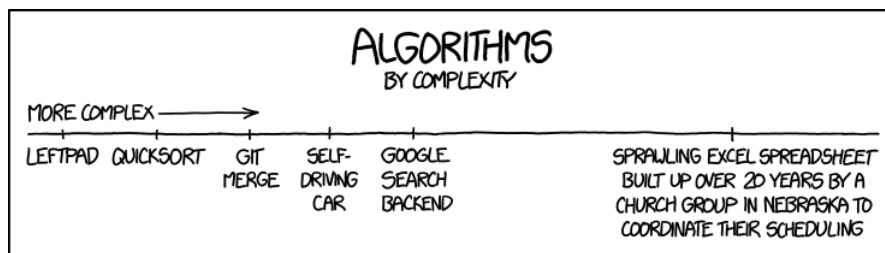


## Chapitre Bonus

# Raisonner sur les programmes



# Table des matières

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction à l'algorithmique</b>                                | <b>1</b>  |
| <b>2</b> | <b>Spécification</b>                                                 | <b>2</b>  |
| <b>3</b> | <b>Correction</b>                                                    | <b>3</b>  |
| 3.1      | Preuves de correction par récurrence . . . . .                       | 3         |
| 3.2      | Preuves de correction avec invariants . . . . .                      | 5         |
| 3.3      | Applications : Correction des algorithmes de tri . . . . .           | 6         |
| 3.3.1    | Tri par insertion . . . . .                                          | 6         |
| 3.3.2    | Tri rapide . . . . .                                                 | 7         |
| 3.3.3    | Tri fusion . . . . .                                                 | 7         |
| <b>4</b> | <b>Terminaison</b>                                                   | <b>8</b>  |
| 4.1      | Preuves avec variants . . . . .                                      | 8         |
| 4.1.1    | Impératif . . . . .                                                  | 8         |
| 4.1.2    | Fonctionnel . . . . .                                                | 9         |
| 4.2      | Interlude sur les relations d'ordre, ordre lexicographique . . . . . | 10        |
| 4.2.1    | Relations sur un ensemble . . . . .                                  | 10        |
| 4.2.2    | Relation d'équivalence . . . . .                                     | 10        |
| 4.2.3    | Relation d'ordre . . . . .                                           | 11        |
| 4.2.4    | Ordre bien fondé . . . . .                                           | 13        |
| 4.2.5    | Retour sur la terminaison des fonctions . . . . .                    | 14        |
| <b>5</b> | <b>Complexité</b>                                                    | <b>16</b> |
| 5.1      | Complexité temporelle . . . . .                                      | 16        |
| 5.1.1    | Formalisme . . . . .                                                 | 16        |
| 5.1.2    | Complexité des boucles . . . . .                                     | 18        |
| 5.1.3    | Complexité des fonctions récursives . . . . .                        | 20        |
| 5.1.4    | Théorème maître . . . . .                                            | 20        |
| 5.2      | Complexité spatiale . . . . .                                        | 21        |
| 5.3      | Complexité amortie . . . . .                                         | 22        |
| 5.3.1    | Formalisme . . . . .                                                 | 22        |
| 5.3.2    | Méthode du potentiel . . . . .                                       | 22        |
| 5.4      | Applications . . . . .                                               | 24        |
| <b>6</b> | <b>Induction structurelle</b>                                        | <b>24</b> |
| 6.1      | Définition d'ensemble inductifs . . . . .                            | 24        |
| 6.2      | Preuve par induction structurelle . . . . .                          | 24        |
| 6.3      | Fonctions sur un ensemble inductif . . . . .                         | 24        |

# 1 Introduction à l'algorithmique

## Définition - Algorithme

Un algorithme est une suite finie non ambiguë d'opérations ou d'instructions élémentaires permettant de résoudre un problème.

On constate alors **2 modèles d'instructions élémentaires** :

### ➤ Impératif :

- Basé sur les machines de Turing
- Déclarations/affectations de variables
- Séquences<sup>1</sup>
- Conditionnelles
- Boucles (non-bornée)

### ➤ Fonctionnel :

- Basé sur le  $\lambda$ -calcul d'Alonzo Church
- Déclarations/affectations de variables
- Fonctions
- Appels de fonctions

Ces deux modèles sont en réalité équivalents :

## Thèse de Church-Turing

Les instructions des 2 modèles suffisent à elles-seules à exprimer TOUS les algorithmes.

## Définition - Système de Turing

Un système qui suffit pour écrire tous les algos est appelé système de Turing.

L'objectif de ce chapitre est de **comparer les programmes** solutions à un même problème :

- **Correction** : L'un est garanti d'être correct, l'autre non (ne renvoie pas le résultat attendu).
- **Terminaison** : L'un est garanti de terminer, l'autre non (ex : boucle infinie)
- **Complexité** : L'un réalise un tel nombre d'opérations élémentaires, l'autre en fait 4 fois plus (ordre de grandeur)

## Définition - Bug

Défaut(s) de conception du programme se manifestant par des anomalies de fonctionnement. Il peut être dû à un dépassement de capacité, des boucles infinies, des cas particuliers imprévus etc. ...

Malgré l'expertise de l'algorithmique, il y a encore de nos jours de sérieux bugs (ex : Robot Spirit sur Mars). Il faut donc limiter les bugs à l'imprévu en prouvant formellement que l'algorithme est correct et termine.

La première étape est donc d'établir les *spécifications* des algorithmes et du problème.

### EXEMPLE 1

Si on se réfère au problème du tri d'une liste, on aura différents algorithmes de tri mais un seul problème : renvoyer la liste triée ou la trier en place. Formalisons donc ces notions.

1. comme en OCAML avec les ";" et en C avec les "}"

## 2 Spécification

### Définition - Spécification d'un problème algorithmique

La spécification d'un problème comporte deux parties :

- La description des entrées admissibles.
- La description du résultat/des effets attendus.

Voyez cela comme un contrat que doit respecter un algorithme qui résout ce problème.

### Définition - Spécification d'un algorithme

La spécification d'un algorithme comporte quatre parties :

- **Entrées** : nombre et type précis.
  - En OCAML, il faut préciser la signature.
  - En C, le prototype est inclus dans les déclarations des fonctions.
- **Préconditions** : une ou plusieurs propriété(s) que doivent vérifier les entrées.
  - Ex :  $n \in \mathbb{N}^*$ , tableau trié
  - Si elles ne sont pas respectées  $\Rightarrow$  *undefined behavior*.  
Pour éviter cela, on utilisera la *Programmation Défensive*.
- **Sortie** : type du résultat (unique)
- **Postconditions** : valeur du résultat + *side effect*<sup>a</sup>

a. Effet qu'a l'algorithme sur son environnement (modifications des variables, affichage, etc...)

On a maintenant le nécessaire pour définir strictement le contrat à respecter pour un algo.

#### EXEMPLE 2. Exponentiation

Considérons le calcul de la  $n$ -ème puissance d'un nombre  $a$  comme tel :

$$a^n = \underbrace{a \times \dots \times a}_{n \text{ fois}}$$

Néanmoins, notre écriture de  $a^n$  n'a de sens que si  $n \in \mathbb{N}$ . On a donc la spécification suivante :

- **Entrées** :  $a$  un nombre,  $n$  un entier
- **Préconditions** :  $a \in \mathbb{R}, n \in \mathbb{N}$
- **Sortie** :  $a^n$
- **Postcondition** : affichage du résultat dans le terminal

Ici, la spécification ne précise qu'une sortie possible, ce qui n'est franchement pas toujours le cas :

#### EXEMPLE 3. Recherche dans un tableau

- **Entrées** : tableau  $t$ , élément  $elt$
- **Préconditions** :  $t$  doit être un tableau d'éléments du même type que celui de  $elt$ .
- **Sortie** :
  - Si  $elt$  est présent dans  $t$  : renvoyer un indice  $i$  tq.  $t[i] = elt$ .
  - Sinon : renvoyer  $-1$ .
- **Postconditions** : entrées non-modifiées.

### Le point vocabulaire

- **Entrée** : pour l'algorithme.
- **Paramètre** : pour l'implémentation de la fonction.
- **Argument** : pour l'appel de la fonction (valeur du paramètre donc).

### 3 Correction

On va donc prendre en compte la spécification d'un algorithme, et regarder si avec ces entrées, il renvoie quelque chose qui respecte les conditions sur la sortie et les postconditions.

#### Définition - Correction partielle

Un algorithme est dit partiellement correct si, quelles que soient les valeurs des entrées : soit il renvoie une sortie conforme à la spécification, soit il ne termine pas.

#### Définition - Correction totale

Un algorithme est dit totalement correct s'il est partiellement correct et qu'il termine.

EXEMPLE 4 Considérons un algorithme pour rechercher la racine carrée d'un nombre :

---

**Algo - Recherche de la racine carrée**

---

**Require:**  $x \in \mathbb{R}$

```
if  $x < 0$  then
  return -1
else
  approx  $\leftarrow x/2$ 
  while approx  $\times$  approx  $\neq x$  do
    approx  $\leftarrow$  (approx +  $x/\text{approx}$ )/2
  return approx
```

---

Bien que la formule de Newton-Raphson soit correcte pour rechercher la racine carrée d'un nombre, la boucle while ne terminera presque jamais à cause de l'approximation des réels sur machine. Dans le cas où l'algo termine, il renvoie bien le résultat attendu.

Voici donc un algorithme partiellement correcte mais pas totalement correct!

On va donc élaborer des techniques pour montrer qu'un algorithme est correct totalement ou non.

#### 3.1 Preuves de correction par récurrence

L'idée ici est que pour justifier la correction d'un algorithme, on a besoin de procéder à un suivi de l'évolution de certaines variables. Ce suivi est, dans certains cas, très simple car on ne modifie aucune variable ni aucune structure de données. On peut alors ramener le raisonnement sur les algorithmes à un raisonnement équationnel similaire à celui qui se présenterait dans un calcul mathématique ordinaire.

Cette situation simplifiée s'applique notamment à de nombreux programmes OCAML puisque dans ce langage, les variables sont *immuables*<sup>2</sup>, de même que certaines structures de base comme les listes.

EXEMPLE 5. Considérons de nouveau le problème de l'exponentiation. Nous allons considérer la version naïve et la version rapide (on détaillera plus tard en quoi elle est plus rapide) :

```
1 let rec expo a n =
2   if n = 0 then 1
3   else a * expo a (n-1)
```

On peut alors résumer l'action de cette version par ces deux équations :

$$\begin{cases} \text{expo } a \ 0 = 1 \\ \text{expo } a \ n = a \times (\text{expo } a \ (n-1)) \quad \text{si } n > 0 \end{cases}$$

```
1 let rec expo_rapide a n =
2   if n = 0 then 1
3   else
4     let tmp = expo_rapide a (n/2) in
5     if n mod 2 = 0 then tmp * tmp
6     else a * tmp * tmp
```

---

2. variable dont la valeur ne peut pas être modifiée une fois qu'elle a été créée

Ici, on garde le même cas de base mais on fait une différenciation des puissances un peu subtile, ce qui donne les équations suivantes :

$$\begin{cases} \text{expo\_rapide } a \ 0 = 1 \\ \text{expo\_rapide } a \ (2k) = (\text{expo\_rapide } a \ k)^2 \quad \text{si } k > 0 \\ \text{expo\_rapide } a \ (2k+1) = a \times (\text{expo\_rapide } a \ k)^2 \end{cases}$$

Montrons alors par récurrence (forte) que `expo_rapide` est correcte :

- Pour  $n \in \mathbb{N}$ , notons  $H_n$  : “`expo_rapide a n` renvoie  $a^n$ ”
- $n = 0$  : cas de base, immédiat.
- $H_0, \dots, H_{n-1} \Rightarrow H_n$  :
  - Si  $n$  est pair : i.e  $\exists k \in \mathbb{N}$  tq.  $n = 2k$ . Alors, par H.R, `expo_rapide a k` renvoie  $a^k$ . Ensuite, d’après notre relation de récurrence, on renvoie  $(a^k)^2 = a^{2k} = a^n$ .
  - Si  $n$  est impair : i.e  $\exists k \in \mathbb{N}$  tq.  $n = 2k + 1$ . Alors, par H.R, `expo_rapide a k` renvoie  $a^k$ . Ensuite, d’après notre relation de récurrence, on renvoie  $a \times (a^k)^2 = a \times a^{2k} = a^{2k+1} = a^n$

$H_n$  vraie. Ce qui clôt la récurrence.

Ainsi, pour tout  $n \in \mathbb{N}$ , `expo_rapide a n` renvoie  $a^n$ . D’où la correction de cet algorithme!

EXEMPLE 6. Plus compliqué, considérons le tri par insertion en OCAML :

```

1 let rec insert x l =
2   match l with
3   | [] -> [x]
4   | y::ys when x <= y -> x::l
5   | y::ys -> y::(insert x ys)
6
7 let rec insertion_sort l =
8   match l with
9   | [] -> []
10  | x::xs -> insert x (insertion_sort xs)

```

On revient donc aux équations suivantes :

$$\text{insert } x \ l = \begin{cases} [x] & \text{si } l = [] \\ x::l & \text{si } l = (y::ys) \text{ et } x \leq y \\ y::(\text{insert } x \ ys) & \text{si } l = (y::ys) \text{ et } x > y \end{cases}$$

et

$$\text{insert\_sort } l = \begin{cases} [] & \text{si } l = [] \\ \text{insert } x \ (\text{insertion\_sort } xs) & \text{si } l = x::xs \end{cases}$$

On a donc besoin de **justifier la correction de la fonction auxiliaire (`insert`) avant de justifier la correction de la fonction principale (`insertion_sort`)**

➤ **Correction auxiliaire :**

- Si  $n \in \mathbb{N}$ , notons  $H_n$  : “Pour une liste triée  $l$  de longueur  $n$ , `insert x l` renvoie la liste triée de longueur  $n + 1$  comprenant exactement les éléments de  $l$  et  $x$ .”
- $n = 0$  : cas de base, immédiat.
- $H_{n-1} \Rightarrow H_n$  : Considérons une liste triée  $l$  de longueur  $n \geq 1$ , alors on peut la représenter comme  $l = y::ys$  avec  $ys$  une liste triée de taille  $n - 1$ .
  - Si  $x \leq y$  : alors on a immédiatement que  $x::y::ys$  est une liste triée.
  - Sinon : Par H.R,  $l' = \text{insert } x \ ys$  est une liste triée dont les éléments sont tous supérieurs ou égaux à  $y$  (car  $l$  était supposée triée et  $x > y$ ) donc  $y::l'$  est une liste triée comprenant exactement les éléments de  $l$  et  $x$ .

$H_n$  vraie, ce qui clôt la récurrence.

On a donc bien justifié la correction de la fonction `insert`.

### ➤ Correction principale

- ❑ Si  $n \in \mathbb{N}$ , notons  $H_n$  : “ Pour une liste  $l$  de longueur  $n$ , `insertion_sort l` renvoie la liste  $l$  triée.”
- ❑  $n = 0$  : cas de base, immédiat.
- ❑  $H_{n-1} \Rightarrow H_n$  : Considérons une liste  $l$  de longueur  $n \geq 1$ .  
Alors, nécessairement,  $l$  peut s’écrire  $l = x : xs$ . Par H.R, `insertion_sort xs` renvoie  $xs$  triée et comme `insert` est une fonction correcte, `insert x (insertion_sort xs)` est la liste  $l$  triée.

$H_n$  vraie, ce qui clôt la récurrence.

D’où la correction de la fonction principale.

Tout ça marche très bien avec des procédés récursifs mais, avec des procédés itératifs, on a développé quelque chose de plus pratique qu’une récurrence : les **invariants**.

En réalité, en plus de formaliser notre raisonnement précédent sur les boucles, les invariants vont nous permettre de raisonner de manière plus puissante, en prenant en compte l’évolution d’une variable/structure (ex : tableau).

## 3.2 Preuves de correction avec invariants

### Définition - Invariant

Un invariant de boucle est un *prédicat*<sup>a</sup> qui, quelles que soient les entrées valides fournies :

- est vérifié avant d’entrer dans la boucle
- s’il est vérifié en entrée d’une itération, il est vérifié en sortie de celle-ci

En général, pour un  $i$ -ème tour de boucle, on le note  $\text{Inv}(i)$ .

<sup>a</sup>. propriété booléenne

**Rmq** : il peut y avoir des invariants intéressants (ex : dire que  $1 = 1$  à chaque boucle n’est pas super utile).

Ainsi, avec les invariants, on peut justifier qu’une propriété est vraie en début de boucle et que comme la boucle est correcte pour cette propriété, cette dernière restera vraie en fin de boucle. Ce qui nous permettra de conclure quant à la correction de certains algs.

### Proposition

Pour toute entrée valide, si l’invariant est valide (i.e vérifié à chaque boucle) alors il est vérifié à la fin de l’exécution de la boucle. On dit alors que c’est un invariant correct.

### Proposition

Si l’invariant de boucle est correct, alors la boucle concernée est correcte (effectue correctement ce qu’elle doit faire : modifier une structure/une variable etc...).

EXEMPLE 7. Considérons l’algorithme itératif pour déterminer selon un entier  $n$ , l’entier  $n!$  :

---

**Algo** - Factorielle

---

```
res ← 1
for i ← 1 to n do
  res ← i × res
return res
```

---

On remarque alors l’invariant suivant :

$\text{Inv}(i)$  : “ `res` =  $(i - 1)!$  ”

DÉMONSTRATION.

- ❑ **Avant la boucle** : `res` =  $1 = (1 - 1)!$
- ❑ **Supposons  $\text{Inv}(i)$  vrai et montrons  $\text{Inv}(i + 1)$**  : On a `res` ←  $i \times \underbrace{\text{res}}_{=(i-1)!}$  d’après  $\text{Inv}(i)$ .

D’où, en fin de boucle  $i$ , i.e en début de boucle  $i + 1$ , on a `res` =  $i!$ .

D’où  $\text{Inv}(i + 1)$ .

Cet invariant est donc correct, et comme l’algo n’est composé que d’une seule boucle, on peut dès lors dire que cet algorithme est correct.

### 3.3 Applications : Correction des algorithmes de tri

#### 3.3.1 Tri par insertion

##### Algorithme - Tri par insertion

```
Require: tab un tableau de taille  $n \in \mathbb{N}^*$   
for  $i \leftarrow 1$  to  $n - 1$  do  
   $v \leftarrow \text{tab}[i]$   
   $j \leftarrow i$   
  while  $j > 0$  &  $\text{tab}[j-1] > v$  do  
     $\text{tab}[j] \leftarrow \text{tab}[j-1]$   
     $j \leftarrow j - 1$   
   $\text{tab}[j] \leftarrow v$ 
```

Le tri par insertion utilisant deux boucles imbriquées, nous allons avoir des invariants pour la boucle externe **for** et la boucle interne **while**. Néanmoins, certains invariants vont se rejoindre, commençons donc par énumérer les invariants pour la boucle extérieure :

Soit  $i \in [1; n - 1]$ ,

1. Pour tout indice  $k_1, k_2 \in [0; i]$  tels que  $k_1 < k_2$ , on a  $\text{tab}[k_1] \leq \text{tab}[k_2]$ .  
Cette première propriété est vérifiée au début du premier tour :  $i$  vaut 1 et le segment  $[0; 1[$  (i.e  $\text{tab}[0]$ ) est évidemment trié car de longueur 1.  
Supposons cette propriété vérifiée en début de boucle, montrons qu'en fin de boucle, le segment  $[0; i + 1[$  est trié.  
En fait, il faut s'intéresser à l'action de la boucle **while** : en supposant qu'elle soit correcte (preuve ci-dessous), cette dernière réalise deux sous-segments en fin d'exec. En effet, elle décale vers la droite tous les éléments du segment  $[0; i[$  qui sont plus grands que  $v$ , afin que l'on puisse insérer  $v$ . Si l'on a supposé  $[0; i[$  trié, alors les deux sous-segments  $[0; j[$  et  $]j; i]$  seront triés, tout en ayant des éléments soit tous inférieurs à  $v$ , soit tous supérieurs. Ainsi, insérer  $v$  en position  $j$  nous permet de dire que le segment  $[0; i] = [0; i + 1[$  est trié.  
Tout cela peut se résumer par 3 invariants supplémentaires :
2. L'indice  $j$  est tel que  $0 \leq j \leq i$
3. Le segment  $\text{tab}[0; i]$  est trié, en ignorant l'élément d'indice  $j$ .
4. Pour tout indice  $k \in ]j; i]$ , on a  $v < \text{tab}[k]$

**Correction de la boucle interne** Montrons que les propriétés 2,3 et 4 sont bien des invariants de la boucle interne **while**. Considérons un tour quelconque de la boucle **for**, prenons  $i \in [1; n - 1]$ . Supposons donc que l'invariant 1 est vérifié au début de cette boucle **for**. Étudions les propriétés 2,3 et 4 :

- Au début de la boucle **while** :
  - La propriété 2 est vraie car  $j$  est initialisé à  $i$ .
  - La propriété 3 est vraie car, quand  $j = i$ , on parle du segment  $[0; i]$ , qui est supposé trié par hypothèse.
  - La propriété 4 est vraie car, quand  $j = i$ , on a  $]j; i] = \emptyset$ .
- Durant un tour interne de la boucle **while** : notons  $j'$  l'état de  $j$  à la fin du tour et  $\text{tab}'$  l'état de  $\text{tab}$  à la fin du tour. Supposons les trois propriétés vérifiées au début du tour.
  - Au début du tour,  $0 < j \leq i$  (on a pas inégalité large à gauche sinon le tour n'aurait pas pu exister), et comme  $j$  décroît de 1 pendant un tour, on a bien  $0 \leq j' \leq i$ .
  - À la fin du tour, pour tout  $k \in [0; j']$ ,  $\text{tab}'[k] = \text{tab}[k]$  et pour tout  $k \in ]j' + 1; i]$ ,  $\text{tab}'[k] = \text{tab}[k]$ .  
Enfin,  $\text{tab}[j' + 1] = \text{tab}[j']$   
Ainsi, soient  $k_1, k_2 \in [0; i]$  tq.  $k_1, k_2 \neq j'$  et  $k_1 < k_2$ .
    - > Si  $k_2 \leq j' - 1$  : alors  $\text{tab}'[k_1] = \text{tab}[k_1]$  et de même pour  $k_2$ . Or, par hyp. en début de tour, on a bien  $\text{tab}[k_1] \leq \text{tab}[k_2]$ . Donc cette partie reste triée.
    - > Si  $k_1 = j' + 1$  : alors  $\text{tab}'[k_1] = \text{tab}[k_1 - 1] \leq \text{tab}[k_2] = \text{tab}'[k_2]$ .
    - > Si  $k_2 = j' + 1$  : alors  $\text{tab}'[k_1] = \text{tab}[k_1] \leq \text{tab}[k_2 - 1] = \text{tab}'[k_2]$

On a donc bien  $\text{tab}'[k_1] \leq \text{tab}'[k_2]$ .

- À la fin du tour, pour tout  $k \in ]j' + 1; i] = ]j, i]$ , on a  $\text{tab}'[k] = \text{tab}[k]$ , donc  $v < \text{tab}'[k]$  (car on a supposé la propriété vraie au début du tour). Enfin,  $v < \text{tab}[j - 1] = \text{tab}'[j] = \text{tab}'[j' + 1]$ . Cette propriété est donc vraie à la fin du tour.



**Conclusion, correction de la boucle principale** On sait maintenant que la boucle interne est correcte grâce aux invariants 2,3 et 4. On en déduit que la propriété 1 est bien préservée par la boucle **for** au cours de son exécution et donc que le tableau final est bien trié.

**Néanmoins, la preuve n'est pas terminée!** En effet, nous avons justifié que le tableau obtenu à la fin était trié mais nous n'avons jamais garanti qu'il comportait exactement les éléments du tableau initial. Pour cela, il suffit juste de modifier légèrement les invariants 1 et 3 :

- 1'. Le segment  $[0; i[$  du tableau **tab** est une permutation du segment  $[0; i[$  du tableau initial.
- 3'. Les segments  $[0; j[$  et  $]j; i[$  du tableau **tab** sont une permutation du segment  $[0; i[$  du tableau initial.

On conclut en vérifiant que ces invariants sont vrais en début de boucle et sont préservés par cette dernière.

D'où la correction recherchée.

### 3.3.2 Tri rapide

### 3.3.3 Tri fusion

## 4 Terminaison

### Définition

On dit qu'un algorithme termine si et seulement si il renvoie une sortie en un nombre fini d'étapes, quelles que soient les valeurs admissibles passées en entrée.

### Proposition

Montrer la terminaison d'un algorithme revient à montrer la terminaison des boucles non-bornées et des fonctions récursives composant cet algorithme.

Avec des algorithmes aussi triviaux que factorielle ou l'exponentiation, on pourrait penser que prouver une terminaison est chose aisée, mais en réalité non ! Déjà, l'exemple 4 nous montre qu'une boucle **while** n'a rien de trivial. On va donc prouver qu'un algorithme termine notamment grâce à un outil : le **variant**.

### 4.1 Preuves avec variants

#### 4.1.1 Impératif

##### Définition - Variant de boucle

Étant donné un algorithme et une boucle de cet algorithme, un *variant* est une fonction de variables de l'algorithme qui :

- ne prend que des valeurs entières positives ou nulles.
- décroît strictement à chaque tour de boucle.

##### Théorème

Une boucle composée d'instructions qui terminent et admettant un variant est une boucle qui termine.

EXEMPLE 8. Reprenons la version itérative de l'algorithme factorielle (ex 7), ré-écrivons le avec une boucle **while** :

#### Algo - Factorielle

```
res ← n
while n > 0 do
  res ← n × res
  n ← n - 1
return res
```

On prend alors le variant, qui ici est assez visible :

$$\text{Variant}(n) = n$$

- Au départ,  $n$  est positif.
- TANT QUE  $n > 0$  :  $n' = n - 1 \geq 0$ , cette quantité décroît donc strictement et est positive ou nulle.

Cette boucle est composée d'instructions qui terminent (affectations) et dispose d'un variant, donc d'après notre théorème : elle termine. D'où la terminaison de notre algorithme factorielle version "boucle while".

##### Méthode

###### (1) Trouver un variant

- > Regarder les variables de la condition d'arrêt de la boucle, regarder les affectations dans la boucle concernant ces variables.
- > En exprimer une quantité positive.
- > S'assurer qu'elle décroît.

###### (2) Montrer qu'il s'agit bien d'un variant

- > Montrer que cette quantité est toujours positive avant la boucle et tant que la condition est vérifiée.
- > Montrer qu'elle est strictement décroissante d'une itération à la suivante.

(1') Montrer que la boucle ne termine pas : trouver une instance contre-exemple où la boucle est en fait infinie.

#### 4.1.2 Fonctionnel

##### Définition - Variant version récursif

Étant donné un algorithme récursif, un *variant* est une fonction des variables des variables de l'algorithme qui :

- ne prend que des valeurs entières positives ou nulles;
- décroît strictement à chaque appel récursif;

⇒ Principe de *dérécursivation* !

##### Théorème

Une fonction récursive composée d'instructions qui terminent et admettant un variant, termine.

On en déduit alors le théorème généralisé :

##### Théorème

Un algorithme dans lequel toutes les boucles non bornées et fonctions récursives sont composées d'instructions qui terminent et d'un variant, termine.

EXEMPLE 9. Prenons l'algorithme de recherche dichotomique, en version récursive :

---

**Algo** - fonction `search_dico_rec(tab, elt, d, f)`

---

**Require:** Tableau d'entiers `tab` trié et de taille `n`, entier `elt`

```
if d > f then
    return -1
else
    mid ← (f + d)/2
    if tab[mid] == elt then
        return mid
    else if tab[mid] > elt then
        return search_dico_rec(tab, elt, d, mid-1)
    else
        return search_dico_rec(tab, elt, mid+1, f)
```

---

et on le lance avec l'appel `search_dico_rec(tab, elt, 0, n-1)` où `n` est la taille du tableau!

On remarque qu'ici, on ne voit pas directement de quantité positive qui décroît strictement, il faut donc jouer avec les paramètres qui varient : `d` et `f`.

On sait qu'à chaque tour, l'une des deux valeurs est modifiée : soit `f` décroît, soit `d` croît. On considèrera alors la différence :

$$\text{Variant}(d, f) = f - d$$

Cette quantité est positive au premier appel (elle vaut `n - 1`), et on vient de justifier que le fait qu'elle reste positive et qu'elle décroît strictement à chaque appel récursif. Il s'agit donc bien d'un variant. En sachant que cette fonction s'arrête quand la différence devient nulle ou négative, on vient de prouver que cette fonction termine!

## 4.2 Interlude sur les relations d'ordre, ordre lexicographique

### 4.2.1 Relations sur un ensemble

#### Définition - Relation $n$ -aire

Considérons un ensemble non vide  $E$ . Pour  $n \in \mathbb{N}$ , on appelle *relation  $n$ -aire* sur  $E$  une partie  $\mathcal{R}$  de  $E^n$ .  
On appelle  $n$  l'*arité* de la relation.

EXEMPLE 10. Sur  $\mathbb{R}$ , on peut prendre par exemple :

$$\mathcal{R} = \{(a, b, c) \in \mathbb{R}^3 \mid a + b = c\}$$

ici,  $(1, 2, 3) \in \mathcal{R}$  mais  $(3, 2, 1) \notin \mathcal{R}$ .

En informatique, on ne s'intéressera qu'aux relations binaires.

#### Définition - $x\mathcal{R}y$

Soient  $E$  un ensemble non vide et  $\mathcal{R}$  une relation binaire sur  $E$ . Soient  $x, y \in E$ . On dit que  $x$  est en relation avec  $y$  si et seulement si  $(x, y) \in \mathcal{R}$ .

On note alors  $x\mathcal{R}y$

Dans la suite, on considérera  $E$  un ensemble non vide et  $\mathcal{R}$  une relation binaire sur  $E$ .

#### Définition - Réflexivité/transitivité/symétrie/antisymétrie

On dit que  $\mathcal{R}$  est :

- **réflexive** ssi :  $\forall x \in E, x\mathcal{R}x$ ;
- **transitive** ssi :  $\forall x, y, z \in E, x\mathcal{R}y \text{ et } y\mathcal{R}z \Rightarrow x\mathcal{R}z$ ;
- **symétrique** ssi :  $\forall x, y \in E, x\mathcal{R}y \Rightarrow y\mathcal{R}x$ ;
- **anti-symétrique** ssi :  $\forall x, y \in E, x\mathcal{R}y \text{ et } y\mathcal{R}x \Rightarrow x = y$ ;

#### Définition - Relation induite

Soit  $F \subset E$  tel que  $F \neq \emptyset$ .

Alors,  $\mathcal{P} = \mathcal{R} \cap F^2$  définit une relation binaire sur  $F$  qui a les mêmes propriétés (au sens des quatres définies ci-dessus) que  $\mathcal{R}$ .

On appelle alors  $\mathcal{P}$  la *relation induite* par  $\mathcal{R}$  sur  $F$ .

### 4.2.2 Relation d'équivalence

#### Définition - Relation d'équivalence

On dit que  $\mathcal{R}$  est une *relation d'équivalence* ssi elle est réflexive, transitive et symétrique.

#### Définition - Classe d'équivalence

Supposons que  $\mathcal{R}$  soit une relation d'équivalence, on prend  $x \in E$  et on définit la *classe d'équivalence* de  $x$  par :

$$[x]_{\mathcal{R}} = \{y \in E \mid x\mathcal{R}y\}$$

**Rmq :** on la note parfois  $\dot{x}$  ou  $\text{Cl}(x)$ .

EXEMPLE 11. On peut par exemple dans un graphe non-orienté  $G = (S, A)$  la relation suivante sur  $S$  :  $\forall x, y \in S$

$$x\mathcal{R}y \Leftrightarrow y \text{ accessible depuis } x$$

Montrons qu'il s'agit d'une relation d'équivalence :

- **réflexivité** :  $\forall x \in S, x$  est évidemment accessible depuis  $x$
- **transitivité** :  $\forall x, y, z \in S$  tels que  $x\mathcal{R}y$  et  $y\mathcal{R}z$ . Si  $y$  est accessible depuis  $x$  et  $z$  est accessible depuis  $y$ , on a bien l'accessibilité de  $z$  depuis  $x$ . D'où  $x\mathcal{R}z$ .
- **symétrie** :  $\forall x, y \in S$  tels que  $x\mathcal{R}y$ . Alors, comme on travaille dans un graphe non-orienté, si  $y$  est accessible depuis  $x$ , on peut parcourir le chemin dans l'autre sens et donc dire que  $x$  est accessible depuis  $y$ . D'où  $y\mathcal{R}x$ .

### Proposition

Supposons que  $\mathcal{R}$  soit une relation d'équivalence, alors pour tout  $(x, y) \in E^2$ , on a soit (1) soit (2) :

- (1)  $y \in [x]_{\mathcal{R}}$  et donc  $[y]_{\mathcal{R}} = [x]_{\mathcal{R}}$
- (2)  $y \notin [x]_{\mathcal{R}}$  et donc  $[y]_{\mathcal{R}} \cap [x]_{\mathcal{R}} = \emptyset$

DÉMONSTRATION.

D'une part, la décomposition  $E = [x]_{\mathcal{R}} \sqcup \overline{[x]_{\mathcal{R}}}$  est immédiate. Il reste donc juste à montrer l'implication du (1) et l'implication du (2) :

- (1) Si  $x \mathcal{R} y$ , alors par transitivité de  $\mathcal{R}$ , tout élément en relation avec  $x$  est en relation avec  $y$  et de même pour éléments en relation avec  $y$  qui seront en relation avec  $x$  (par symétrie et transitivité de  $\mathcal{R}$ ). D'où l'égalité des classes d'équivalences.
- (2) Supposons qu'il existe  $z \in [y]_{\mathcal{R}} \cap [x]_{\mathcal{R}}$ , alors  $x \mathcal{R} z$  et  $y \mathcal{R} z$ . Par symétrie,  $z \mathcal{R} y$  et donc par transitivité :  $x \mathcal{R} y$ , ce qui est absurde car  $y \notin [x]_{\mathcal{R}}$  par hypothèse.

### Proposition (corollaire) - Ensemble quotient

L'ensemble des classes d'équivalences par  $\mathcal{R}$  forme une partition de  $E$ .

Cet ensemble est appelé *ensemble quotient* de l'ensemble  $E$  par la relation  $\mathcal{R}$ , et est noté  $E/\mathcal{R}$ .

DÉMONSTRATION. Raisonnons par double inclusion

-> Soit  $x \in E$ , alors  $x \in [x]_{\mathcal{R}}$  donc  $x \in \bigcup_{z \in E} [z]_{\mathcal{R}}$ .

D'où l'inclusion directe

<- Réciproquement,  $\bigcup_{z \in E} [z]_{\mathcal{R}}$  est évidemment inclus dans  $E$ .

On a donc bien

$$E = \bigcup_{z \in E} [z]_{\mathcal{R}}$$

Maintenant, notre proposition précédente nous permet de garantir le fait que ces classes d'équivalences sont soit confondues, soit sont deux à deux disjointes, ce qui est précisément le résultat recherché.

### 4.2.3 Relation d'ordre

#### Définition - Relation d'ordre

On dit que  $\mathcal{R}$  est une *relation d'ordre* si et seulement si elle est réflexive, transitive et antisymétrique.

Généralement, on préférera la noter  $\leq$  ou  $\preceq$ .

#### Définition - Ensemble ordonné

Un couple  $(E, \leq)$  où  $\leq$  est une relation d'ordre sur  $E$  est appelé *ensemble ordonné*.

EXEMPLE 12. Considérons la relation  $\leq$  sur un graphe  $G = (S, A)$  orienté et acyclique telle que pour tout  $x, y \in S$ , on a :

$$x \leq y \Leftrightarrow y \text{ est accessible depuis } x$$

- **réflexivité** : immédiate.
- **transitivité** : de même que pour que l'exemple 11.
- **antisymétrie** : Soient  $x, y \in S$  tels que  $x \leq y$ . Alors  $y$  est accessible depuis  $x$  dans  $G$ . Alors, si  $y \neq x$ ,  $x$  ne peut être accessible par  $y$  dans  $G$  car sinon on aurait un cycle (absurde car on a supposé  $G$  acyclique). Donc on a bien le fait que si  $y \leq x$ , ce n'est possible que si  $x = y$ . D'où l'antisymétrie.

#### Définition - Ordre strict

Soit  $(E, \leq)$  un ensemble ordonné. L'ordre strict  $<$  associé à  $\leq$  est défini par :

$\forall x, y \in E$ ,

$$x < y \Leftrightarrow \begin{cases} x \leq y \\ x \neq y \end{cases}$$

⚠ **Un ordre strict n'est pas une relation d'ordre!** (il n'est pas réflexif)

### Définition - Prédecesseur et successeur

Soient  $(E, \leq)$  un ensemble ordonné et  $(x, y) \in E^2$ .

On dit que  $x$  est un *prédecesseur* (resp. *successeur*) de  $y$  ssi  $x < y$  (resp.  $y < x$ ).

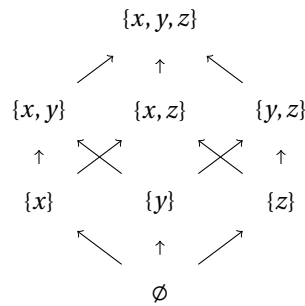
On dit que  $x$  est un *prédecesseur immédiat* (resp. *successeur immédiat*) de  $y$  ssi :

$$\begin{cases} x < y \text{ (resp. } y < x) \\ \nexists z \in E, x < z < y \text{ (resp. } y < z < x) \end{cases}$$

### Définition - Graphe d'un ensemble ordonné

On peut représenter un ensemble ordonné  $(E, \leq)$  par un graphe orienté dont les sommets sont les éléments de  $E$  et tel qu'un arc  $x \rightarrow y$  existe ssi  $y$  est un successeur immédiat de  $x$ .

EXEMPLE 13. Voici le graphe de  $(\mathcal{P}(\{x, y, z\}), \subseteq)$  :



### Proposition

Soient  $(E, \leq)$  un ensemble ordonné et  $x \in E$ .

Alors,

l'ensemble des sommets accessibles depuis  $x$  dans le graphe de  $(E, \leq)$  est l'ensemble  $\{y \in E \mid x \leq y\}$ .

DÉMONSTRATION.

Soit  $y \in E$ . On a que

$$\begin{aligned} y \leq x &\Leftrightarrow x = y \text{ ou } y < x \\ &\Leftrightarrow x = y \text{ ou } x \text{ est un prédecesseur de } y \end{aligned}$$

On montre la dernière équivalence assez facilement, une fois qu'on a établi de proche en proche que si  $x$  est un prédecesseur de  $y$  alors il existe un chemin de prédecesseurs de  $y$  (et donc de successeurs de  $x$ ) les reliant dans le graphe de  $(E, \leq)$ .

### Proposition

Le graphe d'un ensemble ordonné est acyclique.

DÉMONSTRATION.

Supposons qu'il existe un tel cycle (de longueur au moins 2), notons ce cycle (avec  $n \geq 3$ ) :

$$x_1 \rightarrow \cdots \rightarrow x_n = x_1$$

Alors les  $(x_i)_{i \in [2, n-1]}$  sont à la fois des prédecesseurs et des successeurs de  $x_1$ . Cela est absurde car on parle de notion d'ordre strict et ces éléments sont différents de  $x_1$ . Un tel cycle ne peut donc exister.

Ce graphe est donc acyclique.

### Définition - Relation d'ordre totale

Soit  $(E, \leq)$  un ensemble ordonné.

La relation d'ordre  $\leq$  est dite *totale* ssi  $\forall x, y \in E, x \leq y$  ou  $y \leq x$  (sinon, elle est dite *partiellement totale*).

Le cas échéant, on dit alors que  $(E, \leq)$  est un ensemble *totalelement ordonné* (sinon, il est dit *partiellement ordonné*).

#### 4.2.4 Ordre bien fondé

##### Définition - Élément minimal

Soit  $(E, \leq)$  un ensemble ordonné.

On définit un *élément minimal* de  $E$  comme un élément  $m \in E$  tel que  $\forall x \in E$ ,

$$x \leq m \Rightarrow x = m$$

##### Définition - Élément maximal

Soit  $(E, \leq)$  un ensemble ordonné.

On définit un *élément maximal* de  $E$  comme un élément  $M \in E$  tel que  $\forall x \in E$ ,

$$M \leq x \Rightarrow x = M$$

**Rmq** : il n'y a pas toujours unicité! Notamment pour  $(\{2, 3, 6, 9, 12\}, |)$  qui admet comme éléments minimaux 2 et 3.

##### Définition - Minimum

Soit  $(E, \leq)$  un ensemble ordonné.

Un *minimum* pour  $E$  est un élément  $m \in E$  tq.  $\forall x \in E, m \leq x$ .

On parle parfois aussi de *plus petit élément* ou de *minorant*.

##### Définition - Maximum

Soit  $(E, \leq)$  un ensemble ordonné.

Un *maximum* pour  $E$  est un élément  $M \in E$  tq.  $\forall x \in E, x \leq M$ .

On parle parfois aussi de *plus grand élément* ou de *majorant*.

**Rmq** : Ici en revanche, il y a unicité! De plus, si l'ordre est total, être un élément minimal est équivalent à être un minimum.

##### Définition - Ordre bien fondé

Soit  $(E, \leq)$  un ensemble ordonné.

On dit que  $(E, \leq)$  est un *ensemble bien fondé*, ou que  $\leq$  est un ordre bien fondé, ssi  $\forall F \subseteq E$  non vide,  $F$  admet un élément minimal.

Autrement dit,  $E$  est bien fondé si toute partie non vide de  $E$  admet un élément minimal.

En particulier, si  $\leq$  est un ordre total, alors toute partie non vide de  $E$  admet nécessairement un minimum. On parle alors d'*ensemble bien ordonné* et on dit que  $\leq$  est un *bon ordre*.

EXEMPLE 14.

- $(\mathbb{N}, \leq)$  est bien fondé et bien ordonné;
- $(\mathbb{Z}, \leq)$  n'est pas bien fondé :  $\mathbb{Z}$  n'admet pas d'élément minimal;
- $(\mathbb{R}, \leq)$  n'est pas bien fondé :  $\mathbb{R}_+^*$  n'a pas d'élément minimal;
- $(\mathbb{N}, |)$  est bien fondé mais pas bien ordonné;

##### Théorème

$(E, \leq)$  est bien fondé  $\Leftrightarrow$  il n'existe pas de suite infinie strictement décroissante d'éléments de  $E$

DÉMONSTRATION.

- $\Rightarrow$  Supposons par l'absurde que  $(E, \leq)$  soit bien fondé et qu'il existe une telle suite  $(u_n)_n \in E^{\mathbb{N}}$  strictement décroissante. Considérons  $F = \{u_n \mid n \in \mathbb{N}\} \subseteq E$ , non vide. Alors, comme  $(E, \leq)$  est bien fondé,  $F$  admet un élément minimal, noté  $u_{n_0}$  avec  $n_0 \in \mathbb{N}$ . Mais alors, par stricte décroissance de  $(u_n)$ , on aurait  $u_{n_0+1} < u_{n_0}$ , ce qui contredit la minimalité de  $u_{n_0}$ , cela est donc absurde : il ne peut donc exister une telle suite.
- $\Leftarrow$  Raisonnons par contraposée, supposons qu'il existe  $F \subseteq E$  non vide tq.  $F$  n'admette pas d'élément minimal. Soit  $u_0 \in F$ , alors  $u_0$  n'est pas minimal dans  $F$  donc il existe  $u_1 \in F$  tq.  $u_1 < u_0$ . On peut alors raisonner de proche en proche et construire une suite  $(u_n)_n \in E^{\mathbb{N}}$  infinie d'éléments de  $E$  strictement décroissante. D'où l'implication par contraposée.

### Définition - Ordre produit

Soient  $(E_1, \leq_1)$  et  $(E_2, \leq_2)$  deux ensembles ordonnés.

On définit l'ordre produit  $\leq$  sur  $E_1 \times E_2$  par :

$\forall (x_1, x_2), (y_1, y_2) \in E_1 \times E_2$ ,

$$(x_1, y_1) \leq (x_2, y_2) \Leftrightarrow \begin{cases} x_1 \leq_1 y_1 \\ x_2 \leq_2 y_2 \end{cases}$$

### Proposition

Le produit de deux ordres bien fondés est bien fondé.

**Rmq** : le produit de deux ordres n'est pas forcément total, même si les deux ordres dans le produit le sont (prendre le produit de  $(\mathbb{R}, \leq)$  à lui-même :  $(1, 2)$  et  $(2, 1)$  sont incomparables).

### Définition - Ordre lexicographique

Soient  $(E_1, \leq_1)$  et  $(E_2, \leq_2)$  deux ensembles ordonnés.

On définit l'ordre lexicographique  $\leq$  sur  $E_1 \times E_2$  par :

$\forall (x_1, x_2), (y_1, y_2) \in E_1 \times E_2$ ,

$$(x_1, x_2) \leq (y_1, y_2) \Leftrightarrow \left( x_1 <_1 y_1 \text{ ou } \begin{cases} x_1 = y_1 \\ x_2 \leq_2 y_2 \end{cases} \right)$$

### Proposition

L'ordre lexicographique issu de deux ordres bien fondés (resp. totaux) est bien fondé (resp. total).

**Rmq** : On peut généraliser la notion d'ordre produit et d'ordre lexicographique aux  $n$ -uplets.

EXEMPLE 15.

- $(0, 1, 3) \leq (0, 2, 3)$  avec un ordre produit et avec un ordre lexicographique ;
- $(0, 2, 3) \leq (0, 4, 2)$  avec un ordre lexicographique mais pas avec un ordre produit ;
- Dans un dictionnaire, les mots sont rangés dans l'ordre lexicographique ;
- La fonction `strcmp` en C utilise un ordre lexicographique ;

## 4.2.5 Retour sur la terminaison des fonctions

### Un exemple FONDAMENTAL : la fonction d'Ackermann

La fonction d'Ackermann est définie pour  $m, n \in \mathbb{N}$  par :

$$\begin{cases} A(0, n) = n + 1 \\ A(m, 0) = A(m - 1, 1) & \text{si } m \geq 1 \\ A(m, n) = A(m - 1, A(m, n - 1)) & \text{si } m \geq 1 \text{ et } n \geq 1 \end{cases}$$

Si on cherche à prouver la terminaison de cette fonction, on est confronté à un problème :

- > Le premier argument ne décroît pas strictement : un appel à  $A(m, n)$  donne lieu à un appel à  $A(m, n - 1)$  ;
- > Le second argument ne décroît pas strictement non plus : un appel à  $A(m, n)$  donne lieu à un appel à  $A(m - 1, x)$  avec  $x = A(m, n - 1)$  qui n'a aucun raison d'être strictement inférieur à  $n^3$  ;

Mais alors comment faire? Ici, on peut dégainer notre ordre lexicographique sur  $\mathbb{N}^2$ , notons le  $\leq$  : si un appel à  $A(m, n)$  engendre un appel à  $A(m', n')$ , montrons que  $(m', n') < (m, n)$ . Il y a 3 appels engendrés à vérifier :

- $m' = m - 1, n' = 1$  : comme  $m - 1 < m$ , on a bien  $(m', n') < (m, 0)$  selon l'ordre lexicographique ;
- $m' = m, n' = n - 1$  : comme  $m' = m$  et  $n - 1 < n$ , on a bien  $(m', n') < (m, n)$  selon l'ordre lexicographique ;
- $m' = m - 1, n' = x$  (avec  $x = A(m, n - 1)$ ) : comme  $m - 1 < m$ , on a bien  $(m', n') < (m, n)$  selon l'ordre lexicographique.

Ainsi, comme  $(\mathbb{N}, \leq)$  est bien fondé, on a d'après notre proposition ci-dessus que l'ordre lexicographique sur  $\mathbb{N}^2$  est bien fondé. Ce qui signifie, d'après notre théorème, qu'il n'existe pas de suite infinie décroissante d'éléments de  $\mathbb{N}^2$ , quel que soit le sous-ensemble non-vidé de  $\mathbb{N}^2$ . Ainsi, on vient de montrer que **la suite des appels  $(m, n) \in \mathbb{N}^2$  de la fonction d'Ackermann (qui se trouve être strictement décroissante selon l'ordre lexicographique) est finie!**

Les autres instructions terminent trivialement (addition, soustraction etc...), **donc la fonction d'Ackermann termine.**

3. et qui est en fait beaucoup, mais vraiment beaucoup plus grand que  $n!$



## Principe général

### Proposition - Généralisation du variant de boucle

La notion de variant de boucle se généralise en considérant une quantité strictement décroissante dans un ensemble bien fondé.

### Proposition - Terminaison des fonctions récursives

Prouver qu'une fonction récursive termine revient à prouver deux choses :

1. un appel, considéré seul (sans les appels récursifs engendrés), termine toujours ;
2. l'arbre d'appels est fini.

Le premier point est le plus souvent évident, et quand il ne l'est pas, on revient aux techniques vues précédemment (variants de boucle, preuves de terminaison des fonctions auxiliaires appelées).

Pour le second point, il faut montrer que toute suite d'appels récursifs est finie :

- Le cas le plus simple est celui où l'argument est un entier naturel qui décroît strictement au cours des appels ;
- Dans le cas où l'argument n'est pas un entier naturel, on peut souvent se ramener sur  $\mathbb{N}$  en prenant l'image de l'argument par une fonction bien choisie (longueur d'une liste, hauteur de l'arbre, etc...) ;
- Dans les autres cas, il faut munir chaque argument d'une relation d'ordre bien fondée, puis munir l'ensemble des arguments de l'ordre lexicographique ou de l'ordre produit. Ces deux ordres étant alors bien fondés, il n'existe pas de suite d'appels récursifs strictement décroissante et infinie (il faut donc montrer que la suite des appels est strictement décroissante selon l'ordre lexicographique), et donc on peut conclure quant au nombre fini d'appels de la fonction et donc de la terminaison.

EXEMPLE 16. Montrons la terminaison de la fonction  $F : \mathbb{N}^3 \rightarrow \mathbb{N}$  où pour  $a, b, c \in \mathbb{N}$ , on a :

$$\begin{cases} F(0, b, c) = b + c \\ F(a, b, c) = a & \text{si } b = 0 \text{ ou } c = 0 \\ F(a, b, c) = F(a, b - 1, F(a, 0, c - 1)) & \text{si } a = b + c \\ F(a, b, c) = F(a - 1, F(a, b - 1, c + 1), F(a, b, c - 1)) & \text{sinon} \end{cases}$$

Montrons alors les deux points énoncés par la proposition énoncée ci-dessus :

1. Un appel à  $F(a, b, c)$ , considéré seul, n'est constitué que de conditionnelles et d'additions qui terminent.
2. Munissons-nous de l'ordre lexicographique sur  $\mathbb{N}^3$ , considérons un appel à  $F(a, b, c)$  engendrant un appel à  $F(a', b', c')$ , on a alors 2 situations à vérifier :
  - Si  $a = b + c$  : alors  $a' = a, b' = b - 1 < b$  donc d'après l'ordre lexicographique, on a bien  $(a', b', c') < (a, b, c)$  ;
  - Sinon :  $a' = a - 1 < a$  donc d'après l'ordre lexicographique, on a bien  $(a', b', c') < (a, b, c)$  ;

Ainsi, on a bien prouvé les deux points, on peut donc conclure, grâce au fait que  $(\mathbb{N}, \leq)$  soit bien fondé, qu'il n'existe qu'un nombre fini d'appels à  $F$  et que donc  $F$  termine !

## 5 Complexité

On a vu, notamment avec l'exemple de l'exponentiation, que différents algorithmes pouvaient avoir des performances très différentes, tout en résolvant le même problème! La *complexité* est donc l'étude des performances des algorithmes, on l'aborde selon deux critères principaux :

- La *complexité temporelle* décrit le temps de calcul nécessaire à l'exécution de l'algorithme.
- La *complexité spatiale* mesure l'espace mémoire utilisé pour les données de travail de l'algorithme.

Ces notions dépendent bien souvent des entrées.

### 5.1 Complexité temporelle

#### 5.1.1 Formalisme

##### Définition - Complexité temporelle

La *complexité temporelle* d'un algorithme est le nombre d'opérations élémentaires qu'il effectue en fonction de la taille de l'entrée.

Une opération élémentaire peut être :

- une opération arithmétique;
- une affectation;
- une condition (i.e une opération booléenne);
- un accès à un tableau;
- un accès à la tête d'une liste;

EXEMPLE 17.1 Prenons l'exemple de la recherche d'un élément dans un tableau.

---

**Algo** - Recherche d'un élément dans un tableau

---

```
Require: tab de taille n, elt  
for i ← 0 to n do  
    if tab[i] = elt then  
        return i  
return -1
```

---

Ici, on peut déjà encadrer le nombre de comparaisons effectuées, en fonction de la taille du tableau pris en entrée :

- Dans “le meilleur des cas” : l'élément se trouve au début du tableau, on ne fait que une seule comparaison.
- Dans “le pire des cas” : l'élément n'est pas dans le tableau, on le parcourt alors en entier pour s'en rendre compte, on a donc réalisé  $n$  comparaisons.

Ainsi, en notant  $C$  le nombre de comparaisons, on a l'encadrement suivant :

$$1 \leq C \leq n$$

On en déduit de par un encadrement, cette définition :

##### Définition - pire cas / meilleur cas / cas moyen

On distingue alors trois notions pour la complexité d'un algorithme, en fonction d'une taille donnée de l'entrée :

- *Pire cas* : la complexité maximale pouvant être atteinte avec une entrée de la taille donnée. Il s'agit du cas le plus défavorable, sa valeur ne sera à coup sûr jamais dépassée.
- *Meilleur cas* : la complexité minimale pouvant être atteinte avec une entrée de la taille donnée. Cette valeur nous renseigne sur le mieux que puisse faire l'algorithme, sur les entrées les plus favorables.
- *Cas moyen* : la moyenne des complexités pour toutes les entrées possibles d'une taille donnée. Cette valeur donne une indication de ce qu'on peut espérer pour des entrées aléatoires.

##### Définition - Ordre de grandeur et complexité asymptotique

Dans l'étude de la complexité d'un algorithme, on s'intéresse principalement à la *complexité asymptotique* : i.e à la manière dont la complexité évolue pour de grandes valeurs de la taille de l'entrée.

Le plus souvent, on ne cherche pas non plus à avoir une expression exacte de la complexité, mais une simple indication de l'*ordre de grandeur*.

Les notations de Landau donnent différents manières de décrire un ordre de grandeur :

#### Définition - Notations de Landau

Soient une fonction  $f, g : \mathbb{N} \rightarrow \mathbb{N}$ , on a alors les notations suivantes :

➤  $g(n) = \mathcal{O}(f(n))$  si :

$$\exists K \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ tq. } \forall n \geq n_0, g(n) \leq K f(n)$$

⇒ Pire des cas.

➤  $g(n) = \Omega(f(n))$  si :

$$\exists k \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ tq. } \forall n \geq n_0, k f(n) \leq g(n)$$

⇒ Meilleur des cas.

➤  $g(n) = \Theta(f(n))$  si :

$$\exists k_1, k_2 \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ tq. } \forall n \geq n_0, k_1 f(n) \leq g(n) \leq k_2 f(n)$$

⇒ Cas moyen.

On dénote alors quelques fonctions de complexité que l'on croise fréquemment (en notant  $n$  la taille de l'entrée) :

#### Profils de complexité

| Complexité  | Appellation    |
|-------------|----------------|
| 1           | constante      |
| $\log(n)$   | logarithmique  |
| $n$         | linéaire       |
| $n \log(n)$ | linéarithmique |
| $n^2$       | quadratique    |
| $n^3$       | cubique        |
| $2^n$       | exponentielle  |

EXEMPLE 17.2 En reprenant notre exemple, on a alors que

$$C(n) = \mathcal{O}(n) \text{ et } C(n) = \Omega(1)$$

⇒ La complexité de cet algorithme est donc linéaire dans le pire des cas, constante dans le meilleur.

#### Proposition - Règles de calcul sur les $\mathcal{O}$

Soient  $f, g, h_1, h_2 : \mathbb{N} \rightarrow \mathbb{N}$ .

➤ Si  $f(n) = \mathcal{O}(1)$  et  $g(n) = \mathcal{O}(h_2(n))$ , alors

$$f(n) + g(n) = \mathcal{O}(h_2(n)) \text{ et } f(n) \times g(n) = \mathcal{O}(h_2(n))$$

➤ Si  $f(n) = \mathcal{O}(h_1(n))$  et  $g(n) = \mathcal{O}(h_1(n))$ , alors

$$f(n) + g(n) = \mathcal{O}(h_1(n))$$

➤ Si  $f(n) = \mathcal{O}(h_1(n))$  et  $g(n) = \mathcal{O}(h_2(n))$ , alors

$$f(n) + g(n) = \mathcal{O}(h_1(n) + h_2(n))$$

➤ Si  $f(n) = \mathcal{O}(h_1(n))$  et  $g(n) = \mathcal{O}(h_2(n))$ , alors

$$f(n) \times g(n) = \mathcal{O}(h_1(n) \times h_2(n))$$

### 5.1.2 Complexité des boucles

#### ➤ Pour les boucles simples

Généralement, l'entête de la boucle nous donne directement le nombre de tours, il nous suffit juste de voir cela comme une somme!

EXEMPLE 18. Considérons une boucle assez quelconque écrite en langage C :

```
1 for (int i = 0 ; i < n ; i++){
2     // Op. quelconques...
3 }
```

Ainsi, en notant  $C_i$  le nombre d'opérations élémentaires faites au tour de boucle  $i$ , on a que le nombre total d'opérations est :

$$C(n) = \sum_{i=0}^{n-1} C_i$$

Et donc, si  $\forall i \in [0; n-1], C_i = \mathcal{O}(1)$ , on a :

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-1} C_i \\ &= \sum_{i=0}^{n-1} \mathcal{O}(1) \\ &= \mathcal{O}(1) \times \sum_{i=0}^{n-1} 1 \\ &= \mathcal{O}(1) \times n \end{aligned}$$

et  $n$  est tout sauf une constante, considérons alors cela comme un produit entre deux fonctions.

Finalement, selon nos règles de calculs, on a :

$$C(n) = \mathcal{O}(n)$$

 **Méfiez-vous!** Des fois, les  $C_i$  ne seront pas en  $\mathcal{O}(1)$  et disposeront même de “coûts cachés” en faisant appel à des fonctions auxiliaires par exemple.

#### ➤ Pour les boucles conditionnelles

Le nombre de tours d'une boucle **while** est déterminé en comparant la condition d'arrêt et l'évolution des différents éléments intervenant dans cette condition d'arrêt :

```
1 int i = n;
2 while (i > 0){
3     // Op. quelconques ...
4     i --;
5 }
```

On peut alors, dans certains cas, retrouver les mêmes schémas que pour les boucles simples. Néanmoins, ce ne sera pas toujours le cas :

EXEMPLE 19. Considérons le programme suivant :

```
1 int a = 0;
2 int b = 1;
3 while (b < n){
4     b = a + b ;
5     a = b - a ;
6 }
```

Ici, le nombre de tours de boucle est lié au nombre de nombres de Fibonacci dans l'intervalle  $[0; n]$ . Ce nombre peut être calculé, mais cela repose sur une analyse mathématique non triviale (exo).

➤ **Pour les boucles consécutives** on additionnera juste les complexités des deux boucles (cf. règles de calculs).

## ➤ Pour les boucles emboîtées

On calcule d'abord la complexité de l'intérieur de la boucle, puis on se ramène à une somme classique comme pour une boucle simple.

EXEMPLE 20. Considérons la double boucle suivante :

```

1  for (int i = 0; i < n ; i++ ){
2      for (int j = 0; j < n ; j++ ){
3          // Op. quelconques
4      }
5  }
```

Ainsi, pour  $i, j \in \llbracket 0; n-1 \rrbracket$ , on note  $C_{i,j}$  le nombre d'opérations élémentaires réalisées dans la double boucle au tour  $(i, j)$ .

Alors, on a

$$C(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} C_{i,j}$$

Et donc, si  $\forall i, j \in \llbracket 0; n-1 \rrbracket, C_{i,j} = \mathcal{O}(1)$  :

$$\begin{aligned}
 C(n) &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} C_{i,j} \\
 &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathcal{O}(1) \\
 &= \mathcal{O}(1) \times \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 \\
 &= \mathcal{O}(1) \times n^2 \\
 &= \mathcal{O}(n^2)
 \end{aligned}$$

EXEMPLE 21. On rappelle l'algorithme de tri par sélection :

---

**Algo** - Tri par sélection

---

**Require:** `tab` de taille  $n \geq 2$

```

for i ← 0 to n-2 do
    j_min ← i
    for j ← i+1 to n-1 do
        if tab[j] < tab[j_min] then
            j_min ← j
    swap(tab, i, j_min)
```

---

Ainsi, à l'intérieur de la boucle interne, on a que des opérations en  $\mathcal{O}(1)$ . La boucle externe comprend la boucle interne et un appel à la fonction `swap`, supposé en  $\mathcal{O}(1)$  également.

On a donc :

$$\begin{aligned}
 C(n) &= \sum_{i=0}^{n-2} \left( \mathcal{O}(1) + \sum_{j=i+1}^{n-1} \mathcal{O}(1) \right) \\
 &= \sum_{i=0}^{n-2} \left( \mathcal{O}(1) + \mathcal{O}(n-i-1) \right) \\
 &= \mathcal{O}(n^2)
 \end{aligned}$$

Néanmoins, si l'on veut compter exactement le nombre de comparaisons effectuées, on a :

$$\begin{aligned}
 C(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\
 &= \sum_{i=0}^{n-2} (n-i-1) \\
 &= \frac{n(n-1)}{2}
 \end{aligned}$$

### Quelques sommes fréquentes

$$\begin{array}{|l} \sum_{k=0}^n k = \frac{n(n+1)}{2} \\ \sum_{k=0}^n 2^k = 2^{n+1} - 1 \end{array} \quad \left| \quad \begin{array}{l} \sum_{k=0}^n k^2 = \frac{n(n+1)(2n+1)}{6} \\ \sum_{k=1}^n \frac{1}{k} \sim \ln(n) \end{array} \quad \left| \quad \begin{array}{l} \sum_{k=0}^n k^3 = \frac{n^2(n+1)^2}{4} \\ \sum_{k=1}^n \log(k) \sim n \log(n) \end{array} \right.$$

### 5.1.3 Complexité des fonctions récursives

L'astuce systématique est de **se ramener à une formule de récurrence** concernant le nombre d'opérations réalisées pour une certaine taille d'entrée.

EXEMPLE 22. Prenons la version récursive de l'algorithme "factorielle" :

```
1 let rec factorielle n =
2   if n = 0 then 1
3   else n * (factorielle (n-1))
```

Alors, en notant  $C(n)$  le nombre d'opérations élémentaires réalisées par cet algorithme pour l'entier  $n$ , on a :

$$\begin{cases} C(0) = 1 \\ C(n+1) = 1 + C(n) \end{cases}$$

(on oublie pas le coût de la multiplication!)

Ainsi, on retrouve l'expression d'une suite arithmétique, d'où

$$C(n) = n + 1 = \mathcal{O}(n)$$

On retrouve une complexité "pire cas" linéaire.

EXEMPLE 23. Reprenons l'algorithme de la recherche dichotomique dans sa version récursive (exemple 9), on a ainsi (en posant  $n = f - d$ ) :

$$C(n) \leq 1 + C(\lfloor \frac{n}{2} \rfloor)$$

Ici, la technique (ULTRA-CLASSIQUE) est de considérer les entrées de taille de puissance de 2, i.e on prend  $n = 2^k$  avec  $k \in \mathbb{N}$ , alors :

$$C(2^k) \leq 1 + C(2^{k-1})$$

Ainsi, en définissant  $C_k = C(2^k)$ , on a

$$\forall k \in \mathbb{N}, C_k = C(2^k) \leq k$$

Finalement, en passant au log, on a :

$$C(n) \leq \log_2 n$$

d'où  $C(n) = \mathcal{O}(\log(n))$

**Rmq** : Ici, la majoration est faite par  $1 + C(\lfloor \frac{n}{2} \rfloor)$  mais parfois la majoration sera moins évidente à trouver, il faudra d'abord trouver les complexités des fonctions auxiliaires (ex : tri fusion)...

EXEMPLE 24. On retrouve la même complexité pour l'exponentiation rapide grâce au même procédé :

$$C(n) = \mathcal{O}(\log n)$$

### 5.1.4 Théorème maître

#### Théorème (HP) - Théorème maître (avec notation de Landau)

Supposons la relation de récurrence suivante :

$$C(n) = aC(\lfloor \frac{n}{b} \rfloor) + \mathcal{O}(n^\alpha)$$

On a :

- Si  $\alpha < \log_b a$  alors  $C(n) = \mathcal{O}(n^{\log_b a})$
- Si  $\alpha = \log_b a$  alors  $C(n) = \mathcal{O}(n^\alpha \log_b n)$
- Si  $\alpha > \log_b a$  alors  $C(n) = \mathcal{O}(n^\alpha)$

## 5.2 Complexité spatiale

## 5.3 Complexité amortie

### 5.3.1 Formalisme

#### Définition - Complexité amortie

La *complexité amortie* d'un algorithme est une analyse de complexité lissée sur une séquence d'invocations successives de cet algorithme.

EXEMPLE 24. Dans la structure *union-find*, lorsque l'on optimise la fonction `find` en utilisant la compression des chemins : on garde la même complexité pire cas au départ, mais après une série d'appels successifs on peut dire que *son coût a été amorti* (on a raccourci le chemin entre les éléments et leur représentant).

**Rmq :** On utilise notamment la complexité amortie pour des algorithmes regroupant les 3 caractéristiques suivantes :

- avoir une faible complexité sur de nombreuses entrées ;
- être potentiellement très coûteux sur certaines entrées ;
- être tel que, dans toute utilisation répétée de l'algorithme, les entrées coûteuses surviennent suffisamment rarement pour que le coût moyen reste faible ;

### 5.3.2 Méthode du potentiel

Pour conduire une analyse de complexité amortie, la *méthode du potentiel*, aussi appelée *méthode du physicien*, consiste à associer à chaque entrée possible un nombre appelé *potentiel*. Comme en physique, ce « potentiel » dénote quelque chose (ici un coût) qui est latent dans l'entrée, i.e présent mais pas encore réalisé.

#### Définition - Potentiel

Une *fonction de potentiel* est une application  $\Phi$  qui associe à chaque valeur d'entrée possible  $x$  d'un algorithme, une valeur positive ou nulle  $\Phi(x)$ , alors appelée *potentiel*.

➤ Le comportement typique d'un algorithme ayant de bonnes propriétés de complexité amortie alterne entre des opérations de faible coût faisant monter progressivement le potentiel, et des opérations de coût élevé associées à une brusque diminution du potentiel (si la fonction de potentiel nous indique qu'une entrée a potentiellement un faible coût).

Dans l'analyse de complexité amortie, au lieu de ne s'intéresser qu'au coût effectif de chaque opération, on tient compte également de la *variation de potentiel*.

#### Définition - Coût amorti

Considérons une opération  $op$ , dont l'exécution produit une sortie  $x_s$  à partir d'une entrée  $x_e$ .

Le *coût réel* de  $op$ , noté  $C$ , est la complexité temporelle de son exécution.

Son *coût amorti*, noté  $A$ , y ajoute la variation de potentiel de l'entrée à la sortie :

$$A = C + \Phi(x_s) - \Phi(x_e)$$

➤ On peut montrer que, sur une séquence d'opérations enchaînées commençant avec un potentiel nul, le coût réel ne dépasse jamais le coût amorti.

➤ Ce qui caractérise une « séquence d'opérations consécutives » est que chaque nouvelle opération prend comme entrée la sortie de l'opération précédent. Autrement dit, chaque opération repart de l'état, et donc du potentiel, laissé par la précédente.

#### Théorème - Amortissement

Considérons une succession de  $n$  opérations :

$$x_0 \xrightarrow{op_1} x_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} x_n$$

à partir d'une entrée  $x_0$  pour laquelle  $\Phi(x_0) = 0$ .

En notant  $C_i$  le coût réel de  $op_i$  et  $A_i$  son coût amorti, on a alors la relation suivante :

$$\sum_{i=1}^n C_i \leq \sum_{i=1}^n A_i$$



DÉMONSTRATION.

On a, par définition, pour tout  $i \in \llbracket 1; n \rrbracket$  :

$$A_i = C_i + \Phi(x_i) - \Phi(x_{i-1})$$

Donc, en sommant, on obtient une somme télescopique :

$$\begin{aligned} \sum_{i=1}^n A_i &= \sum_{i=1}^n (C_i + \Phi(x_i) - \Phi(x_{i-1})) \\ &= \sum_{i=1}^n C_i + \underbrace{\Phi(x_n)}_{\geq 0} - \underbrace{\Phi(x_0)}_{=0} \end{aligned}$$

D'où

$$\sum_{i=1}^n A_i \geq \sum_{i=1}^n C_i$$

EXEMPLE 25. Considérons une pile munie des opérations suivantes :

- PUSH( $S, x$ ) : empile un objet  $x$  sur la pile  $S$ ;
- POP( $S$ ) : dépile le sommet de la pile  $S$  et retourne l'objet dépilé;
- MULTIPOP( $S, k$ ) : dépile au plus  $k$  objets de la pile  $S$ ;

On peut par exemple voir l'opération MULTIPOP comme ci-dessous :

---

**Algo** - MULTIPOP( $S, k$ )

---

**while**  $S \neq \emptyset$  **and**  $k \neq 0$  **do**

$x \leftarrow \text{POP}(S)$

$k \leftarrow k - 1$

---

Calculons la complexité de chacune des 3 opérations et déduisons-en (avec la méthode du potentiel) le coût amorti pour une suite de  $n$  opérations PUSH, POP et MULTIPOP sur une pile initialement vide.

➤ Les opérations PUSH et POP se font en  $\mathcal{O}(1)$  et MULTIPOP en  $\mathcal{O}(\min\{|S|, k\})$ .

➤ On définit la fonction de potentiel  $\Phi$  comme étant le nombre d'objets présents dans la pile. Ainsi, pour la pile vide (et donc pour  $x_0$  l'entrée initiale), on aura bien  $\Phi(x_0) = 0$ .

D'autre part, comme le nombre d'objets dans la pile est toujours positif, on aura bien  $\Phi(x) \geq 0$  pour tout  $x$  : il s'agit donc bien d'une fonction de potentiel.

Calculons les différences de potentiel en fonction des différentes opérations :

- PUSH : la différence de potentiel après une opération PUSH est  $\Phi(x_{i+1}) - \Phi(x_i) = (|S|_i + 1) - |S|_i = 1$ . On en déduit un coût amorti :

$$\begin{aligned} A_{i+1} &= C_{i+1} + \Phi(x_{i+1}) - \Phi(x_i) \\ &= \underbrace{C_{i+1}}_{=1} + 1 \\ &= 2 \end{aligned}$$

- POP : la différence de potentiel après une opération POP est  $\Phi(x_{i+1}) - \Phi(x_i) = (|S|_i - 1) - |S|_i = -1$ . On en déduit un coût amorti :

$$\begin{aligned} A_{i+1} &= \underbrace{C_{i+1}}_{=1} - 1 \\ &= 0 \end{aligned}$$

- MULTIPOP( $S, k$ ) : la différence de potentiel après une opération MULTIPOP( $S, k$ ) est  $\Phi(x_{i+1}) - \Phi(x_i) = -k'$  avec  $k' = \min\{|S|, k\}$ . On en déduit un coût amorti :

$$\begin{aligned} A_{i+1} &= \underbrace{C_{i+1}}_{=k'} - k' \\ &= 0 \end{aligned}$$

➤ **Le coût amorti de chacune de ces opérations est en  $\mathcal{O}(1)$** , ce qui ne change rien pour les deux premières vis à vis de la complexité pire cas, mais cela change beaucoup pour MULTIPOP !

➤ On a donc un coût total amorti en  $\mathcal{O}(n)$  pour  $n$  opérations consécutives.

#### **5.4 Applications**

### **6 Induction structurelle**

#### **6.1 Définition d'ensemble inductifs**

#### **6.2 Preuve par induction structurelle**

#### **6.3 Fonctions sur un ensemble inductif**